

Database Systems

Fundamentals and Design



Core Components



DBMS Functions



Relational Model



System Architecture



Data Models



Object-Oriented Model

Core Components of Database Systems

Four essential elements working together to form complete systems



Database

Stores all data in an organized manner.



Example: Student records, employee details, sales data



DBMS

Software that manages access to the database.



Example: MySQL, Oracle, SQL Server



Data

The actual information stored in the database.



Example: Names, IDs, salaries, grades



Application

User interface that interacts with the DBMS.



Example: Student management system, payroll software



The components work together to form a complete database system

Database System Architecture Components

Five key elements working together with real-world examples



Hardware

Physical devices used to store and access the database.

 Examples: Servers, hard drives, storage systems



Software

Programs that control and manage the database.

 Examples: MySQL, Oracle, SQL Server



Data

Information stored in the database.

 Examples: Student records, sales transactions



Procedures

Rules for designing and using the database.

 Examples: Backup schedules, security protocols



People

Individuals who interact with the database.

 Examples: DBAs, developers, end users

Real-world Examples



Banking

Data includes account numbers and balances. Security is critical.



Healthcare

Patient records and medical histories require privacy protection.



Education

Student information systems track enrollments and grades.



Hardware



Software



Data



People



Procedures

DBMS Functions and Roles

Ten major functions that ensure efficient data management



Data Storage & Retrieval

Stores and retrieves data efficiently.

💡 Example: Student records by ID



Data Dictionary

Maintains metadata about data structure.

💡 Tracks tables and data types



Security Management

Controls access with authentication.

💡 Access control for HR data



Integrity Management

Ensures data accuracy and consistency.

💡 Prevents duplicate IDs



Backup & Recovery

Protects against data loss.

💡 Restores data after system crash



Concurrency Control

Manages simultaneous data access.

💡 Prevents conflicts in bank transfers



Communication Interface

Enables data sharing over networks.

💡 Web app accessing database API



Transaction Management

Ensures ACID properties.

💡 Maintains consistency in transfers



Data Independence

Separates data structure from applications.

💡 Changing table structure without code



Query Processing

Optimizes and executes user queries.

💡 SQL SELECT statement execution



These functions work together to provide a reliable and efficient data management system

Declarative Query Languages

Specifying what data is needed, not how to get it

What is Declarative?

A declarative query language allows users to state what data they want from the database without specifying how to get it.

The DBMS query processor automatically figures out the best retrieval method.

Example Database

University System with tables:

- Students(student_id, first_name, last_name, gender, age, city)
- Courses(course_id, course_name, credits)
- Enrollments(enroll_id, student_id, course_id, grade)

SQL Query Examples

List all students

```
SELECT * FROM Students;
```

Find students aged above 20

```
SELECT first_name, last_name FROM  
Students WHERE age > 20;
```

Show course names

```
SELECT course_name FROM Courses;
```

Combined clauses example

```
SELECT course_id, COUNT(student_id) AS  
total_enrolled  
FROM Enrollments  
GROUP BY course_id  
HAVING COUNT(student_id) > 5  
ORDER BY total_enrolled DESC;
```

★ Benefits of Declarative Queries

- ✓ Simple and readable syntax
- ✓ Focus on what data is needed
- ✓ Automatically optimized by DBMS
- ✓ Reduces development time and errors

Data Models Comparison

Six data models with internal architectures and storage methods

Data Model		Structure Type	Description	Example	Internal Structure	Storage Method
	Hierarchical	Tree Structure	Data is organized in a parent–child hierarchy	File systems, IMS database	Uses pointers and parent-child trees	Sequential file storage
	Network	Graph Structure	Data records are linked using pointers	CODASYL DBMS	Uses record types and set types to connect records	Linked lists and pointers
	Relational	Table Structure	Data is stored in tables with rows and columns	MySQL, Oracle, SQL Server	Uses tables, indexes, and B-trees	Stored as files of tuples with indexes
	Object-Oriented	Object Structure	Data represented as objects with attributes and methods	db4o, ObjectDB	Combines data and behavior	Object serialization and class structures
	Document/NoSQL	JSON/BSON Document Structure	Stores semi-structured data as key–value pairs or documents	MongoDB, CouchDB	Stores documents in JSON or BSON format	Stored as collections of documents
	Graph	Node–Edge Structure	Data stored as nodes (entities) and edges (relationships)	Neo4j, Amazon Neptune	Uses adjacency lists or adjacency matrices	Node and edge tables for relationships

Data Modeling Principles and Notations

Entity-Relationship Diagrams and Modeling Best Practices

ER Diagram Components



Entities

Real-world objects (tables)

Example: Student, Course



Attributes

Properties (columns)

Example: student_id, name



Relationships

Connections between entities

Example: Enrolls, Teaches



Primary Keys

Unique identifiers

Underlined in ERDs

Modeling Principles



Clarity

Use simple, consistent symbols



Consistency

Follow uniform naming and relationships



Completeness

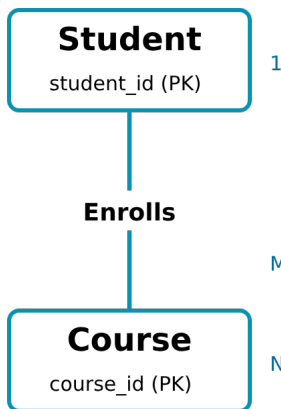
Include all necessary entities and relationships



Cardinality

Show how entities relate (1-M, M-N)

ER Diagram Example



University Database:

- Student (1) → Enrolls (M)
- Course (1) → Enrolls (N)
- M:N Relationship



ER diagrams provide a visual way to understand data flow and relationships

Relational Data Model Fundamentals

The foundation of SQL databases like MySQL and Oracle

Definition

The Relational Data Model organizes data into tables (relations) made up of rows and columns. It was introduced by E. F. Codd in 1970.

Keys

- **Primary Key:** Unique identifier for each record
- **Foreign Key:** Links one table to another
- **Candidate Key:** Unique identifier not chosen as primary

Integrity Rules

- **Entity Integrity:** Each table must have a unique primary key
- **Referential Integrity:** Foreign key must refer to existing record
- **Domain Integrity:** Values must be valid within their domain

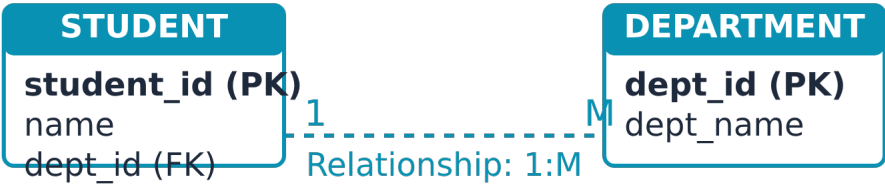
Structure of a Relation

Term	Meaning	Example
Relation (Table)	A collection of related data.	STUDENTS
Tuple (Row)	A single record in a table.	(101, Ali, 20, CS)
Attribute (Column)	A property or field of the table.	student_id, name
Domain	Set of valid values for an attribute.	Age: 18–60

Relationships

- **One-to-One (1:1):** Student ↔ Student ID Card
- **One-to-Many (1:M):** Department → Students
- **Many-to-Many (M:N):** Students ↔ Courses

Example Schema



Applying Relational Modeling Concepts

Six-step process for creating logical database designs with integrity



1. Identify Entities

Determine main objects or concepts in the system.

💡 Example: Student, Department, Course



2. Define Attributes

List properties that describe each entity.

💡 Example: Student (student_id, name, age)



3. Determine Relationships

Identify how entities are related.

💡 Example: Student enrolls in Course



4. Assign Keys

Choose Primary Keys and Foreign Keys.

💡 Example: student_id (PK), dept_id (FK)



5. Apply Constraints

Enforce data rules for integrity.

💡 Example: dept_id must exist in Department

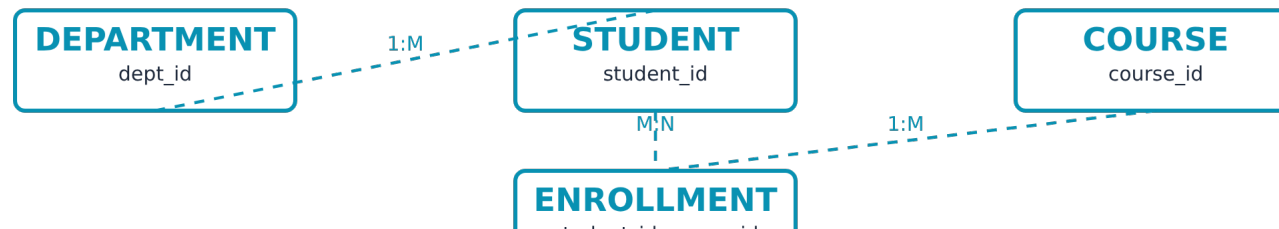


6. Normalize Data

Remove redundancy and organize tables.

💡 Example: Split into STUDENT, DEPARTMENT tables

University Database Example



Object-Oriented Data Model

Objects combining data and behavior with inheritance, encapsulation, and polymorphism

Object Structure



Identity (OID)
Unique object identifier



State
Data or attributes



Behavior
Methods or operations

Key Concepts



Encapsulation

Hides internal data and allows access through methods only



Inheritance

Allows classes to inherit attributes and methods from other classes



Polymorphism

Same method can behave differently for different objects



Relationships

Defines associations among objects (e.g., Student "enrolls in" Course)

Example

University System

Class: Student

Attributes: student_id, name, age

Methods: enroll(), calculateGPA()

Class: Course

Attributes: course_id, course_name, credits

Methods: registerStudent(), displayCourseInfo()

Class: GraduateStudent (inherits from Student)

Attributes: thesis_title

Methods: submitThesis()

Benefits

- ✓ Closer to real-world modeling
- ✓ Promotes reusability through inheritance
- ✓ Data and behavior combined (encapsulation)
- ✓ Supports complex data types